

CFD 2025-26 – Lab 04 – solution

The full code for this lab is provided in the notebook `CFDlab04.ipynb`. The notebook also contains an introductory tutorial on direct/iterative solvers and preconditioning for a scalar elliptic problem (the same that you find in `CFDlab03.ipynb`, but in this lab we go further on preconditioning...). In particular, we highlight that the default parameters of a `LinearVariationalSolver` can be extracted from the predefined dictionary `firedrake.petsc.DEFAULT_KSP_PARAMETERS`.

Moreover, we point out that all `LinearVariationalSolvers` (at least when using PETSc as a linear algebra backend library, as we do throughout all this course) wrap the same PETSc Krylov-type solver `KSP`.¹ For this reason, if we want to apply a *direct* solver, we need to “tweak” the parameters not to actually perform iterations: this can be done by setting `'ksp_type': 'preonly'`, so that only the preconditioning step is applied, and choosing as a preconditioner the direct solver of interest, via the parameter `pc_type`.

Exercise 1

- 1 Following the lines of the previous lab sessions, we first derive the weak formulation of the problem.

We point out that, since the problem has *fully Dirichlet* boundary conditions, an additional constraint on pressure correction is needed. We define the velocity and the pressure space as follows:

$$V_0 := H_0^1(\Omega), \quad V_D := \left\{ \mathbf{v} \in H(\Omega) \mid \mathbf{v}|_{\Gamma^{up}} = \mathbf{i}, \mathbf{v}|_{\partial\Omega \setminus \Gamma^{up}} = \mathbf{0} \right\}, \quad Q := L_0^2(\Omega).$$

and we introduce the lifting $\mathcal{R}\mathbf{g} \in V_D$, where $\mathbf{g} : \partial\Omega \rightarrow \mathbb{R}^2$ denotes the boundary data over the whole of $\partial\Omega$. Testing the system against test functions and integrating by parts, we obtain the following weak problem:

$$\begin{aligned} &\text{Find } \mathbf{u} = \tilde{\mathbf{u}} + \mathcal{R}\mathbf{g} \in V_D, \text{ with } \tilde{\mathbf{u}} \in V_0, \text{ and } p \in Q \text{ s.t.} \\ &a(\tilde{\mathbf{u}}, \mathbf{v}) + b(\mathbf{v}, p) - b(\tilde{\mathbf{u}}, q) = -a(\mathcal{R}\mathbf{g}, \mathbf{v}) + b(\mathcal{R}\mathbf{g}, q) \quad \text{for all } (\mathbf{v}, q) \in V_0 \times Q, \\ &\text{where} \quad a(\mathbf{u}, \mathbf{v}) := \int_{\Omega} (\nabla \mathbf{u} : \nabla \mathbf{v}) \, d\mathbf{x}, \quad b(\mathbf{v}, q) := - \int_{\Omega} \operatorname{div} \mathbf{v} \, q \, d\mathbf{x}. \end{aligned}$$

Introducing the bases $\{\varphi_i\}_{i=1}^{N_V}$ and $\{\psi_\ell\}_{\ell=1}^{N_Q}$ for the FE discrete spaces $V_{0,h} \subset V_0$ and $Q_h \subset Q$, respectively, the following matrices and vectors can be defined:

$$\begin{aligned} A_{ij} &= a(\varphi_j, \varphi_i), & B_{\ell i} &= b(\varphi_i, \psi_\ell), & \mathbf{f}_i &= -a(\mathcal{R}\mathbf{g}, \varphi_i) & \mathbf{f}_{\operatorname{div}, \ell} &= -b(\mathcal{R}\mathbf{g}, \psi_\ell) \\ && \text{for } i, j &= 1, \dots, N_V, & \ell &= 1, \dots, N_Q. \\ \Sigma &= \begin{bmatrix} A & B^T \\ -B & 0 \end{bmatrix} \in \mathbb{R}^{(N_V+N_Q) \times (N_V+N_Q)}, & \mathbf{F} &= \begin{bmatrix} \mathbf{f} \\ \mathbf{f}_{\operatorname{div}} \end{bmatrix} \in \mathbb{R}^{(N_V+N_Q)}, \end{aligned}$$

¹See <https://petsc.org/main/manual/ksp/> for further information. In this course we cover the main functionalities of PETSc, but there are many more!

and the Finite Element Method for the approximation of the problem can be expressed in the following algebraic form:

$$\text{find } \mathbf{U} \in \mathbb{R}^{N_V}, \mathbf{P} \in \mathbb{R}^{N_Q} \quad \text{such that} \quad \Sigma \begin{bmatrix} \mathbf{U} \\ \mathbf{P} \end{bmatrix} = \mathbf{F}.$$

N.B. Notice that, if $\mathcal{R}g$ is such that $\text{div } \mathcal{R}g = 0$, then $\mathbf{f}_{\text{div}} = \mathbf{0}$ and the continuity equation is unperturbed. From now on, we will always make the assumption that the lifting is divergence-free.

To assemble the system block matrix, we proceed as in the previous lab session, defining the finite element spaces ($\mathbf{V}$, $\mathbf{Q}$, and the mixed space $\mathbf{W}$) and functions over a `UnitSquareMesh`. To account for pressure non-uniqueness in the fully Dirichlet case, we also create the null space of the system matrix Σ :

```
nsp = MixedVectorSpaceBasis(
    W, [W.sub(0), VectorSpaceBasis(constant=True)]
)
```

We then assemble the problem by passing the forms and boundary conditions to `LinearVariationalProblem`:

```
wh = Function(W)
vpb = LinearVariationalProblem(a, L, wh, bcs)
```

- 2 To solve the problem with the prescribed parameters, we create a `LinearVariationalSolver` including the problem and the null space created at the previous point:

```
parameters = {'ksp_type': 'gmres', 'pc_type': 'none',
              'ksp_rtol': 1.e-8, 'ksp_max_it': 10000}
solver = LinearVariationalSolver(vpb, nullspace=nsp,
                                solver_parameters=parameters)
```

Then we solve the problem, and in order to investigate the performances of our solver we inspect both the number of iterations and actual computational time required for the solution. The former is accessible from the `solver` via its underlying PETSc “engine” `solver.snes.ksp`, while for the latter we need to import the `time` python module and use its `perf_counter`, which stores the current time at which we are when it is called:

```
from time import perf_counter
t0 = perf_counter()
solver.solve()
print('elapsed time = ', perf_counter() - t0, 's' - # iter = ', \
      solver.snes.ksp.getIterationNumber())
```

Solving the problem with $n = 10$, we notice that a very large number of linear solver iterations (9054) are required, and for finer meshes 10000 iterations are not enough to

preconditioner	$n = 10$	$n = 20$	$n = 40$	$n = 80$
none	9054	–	–	–
iLU	40	118	383	–
block-diagonal (point 3)	47	49	49	49
block-triangular (point 5)	20	21	21	21

Table 1: GMRES iterations required to solve the problem with different preconditioning choices (not reported if exceeding 10'000).

reach convergence.² Indeed, the cavity problem has a strong singularity in the pressure distribution (as we will see at the end of the next point), therefore its solution can be very expensive if approached naively.

Note that the fact that convergence is not reached by the linear solver is signaled by the following error message:

```
ConvergenceError: Nonlinear solve failed to converge after 0
nonlinear iterations.
Reason:
DIVERGED_LINEAR_SOLVE
```

Starting from the bottom, the message **DIVERGED_LINEAR_SOLVE** should not be interpreted as the solution having *diverged* (namely reached values close to the maximum representable numbers in the machine, or contained NaN values), but rather as the solver **not reaching convergence** in the prescribed maximum number of iterations. Regarding the first line of the error, we must mention that for Firedrake every problem is solved in its *nonlinear solver* framework, which however runs for 0 **nonlinear iterations** when it recognizes that the problem is actually linear.

Inspired by possible previous knowledge on elliptic problems, one could think to address this issue by employing preconditioning in the iterative solver, applying the pre-defined preconditioners of PETSc on the monolithic matrix Σ (see <https://petsc.org/main/manual/ksp/#sec-pc> for a list of PETSc pre-defined preconditioners). For example, we could use an incomplete LU factorization as a preconditioner, by changing the parameter **pc_type** as follows:

```
parameters.update({'pc_type': 'ilu'})
solver = LinearVariationalSolver(vpb, nullspace=nsp,
    solver_parameters=parameters)
```

This choice improves the computational efficiency of the solution, but is not enough to solve the problem for increasingly refined mesh, as shown in Table 1.

3 In order to address the issue raised in the previous point, we exploit the structure of

²The number of iterations that the solution takes can vary slightly from one computer to another, especially in the case of ill-conditioned linear systems, due to the slight differences in the generation of machine-precision errors. The computational time measured by **perf_counter** is even more fluctuating, since it is highly sensitive to the current load of the computer. Yet, all comments reported here hold in any case.

the matrix Σ and we employ a block-diagonal preconditioner in which the first block is the stiffness matrix A and the second one is the scaled pressure mass matrix $\frac{1}{\nu}M_p$ (since $\nu = 1$ in this case, we neglect this factor in the implementation), the latter being a good preconditioner for the Schur complement (we will discuss it in the next lab). To implement this preconditioner, we observe that it is the algebraic representation of the following bilinear form:

$$\mathcal{P} : (V_h \times Q_h) \times (V_h \times Q_h) \rightarrow \mathbb{R}, \quad \mathcal{P}((\mathbf{u}, p), (\mathbf{v}, q)) = \int_{\Omega} \nabla \mathbf{u} : \nabla \mathbf{v} \, dx + \int_{\Omega} \frac{1}{\nu} p q \, dx.$$

This can be exploited in Firedrake, since we can provide a preconditioner in the creation of a `LinearVariationalProblem` through its corresponding “preconditioner form” $\mathcal{P}$:

```
pc_form = inner(grad(u), grad(v)) * dx + p * q * dx
vpb = LinearVariationalProblem(a, L, wh, bcs=bcs, aP=pc_form)
```

Then, we can tell the `LinearVariationalSolver` to use this preconditioner by properly setting the solver parameters, as follows:

```
parameters = {'ksp_type': 'gmres',
              'ksp_rtol': 1.e-8, 'ksp_max_it': 10000,
              'pc_type': "fieldsplit",
              'pc_fieldsplit_type': "additive",
              'fieldsplit_0_ksp_type': "preonly",
              'fieldsplit_0_pc_type': "lu",
              'fieldsplit_1_ksp_type': "preonly",
              'fieldsplit_1_pc_type': "lu"}
solver = LinearVariationalSolver(vpb, nullspace=nsp,
                                solver_parameters=parameters)
```

Specifically:

- The `fieldsplit` preconditioner type tells the solver to exploit the block structure of the system, which is given by the structure of the function space used to define the system: typically for (Navier-)Stokes problems, we use a `MixedFunctionSpace` with 2 blocks, corresponding to velocity and pressure, thus defining a block matrix Σ with 2x2 blocks. We will extensively discuss and employ this type of preconditioners in the rest of the course. Detailed information on `fieldsplit` preconditioning can be found in several points of the PETSc documentation.
- The `additive` fieldsplit type informs the solver that we want to use the diagonal blocks of the preconditioner. The name ‘*additive*’ comes from the literature on a specific class of block-diagonal preconditioners: the domain-decomposition Schwarz preconditioners. For our purposes, we will just have to keep in mind that, in PETSc’s fieldsplit framework ‘*additive=block-diagonal*’ and ‘*multiplicative=block-triangular*’ (the latter will be useful in point 5 of the exercise). In other libraries, you may also find the naming ‘*block-Jacobi=block-diagonal*’ and ‘*block-Gauss-Seidel=block-triangular*’.
- Since preconditioning means to solve the linear system corresponding to the precon-

ditioner, and $P^{-1} = \begin{bmatrix} P_0 & 0 \\ 0 & P_1 \end{bmatrix}^{-1} = \begin{bmatrix} P_0^{-1} & 0 \\ 0 & P_1^{-1} \end{bmatrix}$, we need to tell the solver how we want to solve the linear systems corresponding to the blocks P_0 and P_1 . Now, for each of them, we can employ all the parameters that we have already seen before for the monolithic system, simply prepending `fieldsplit_0_` or `fieldsplit_1_` to the name of the parameter. In this case, we apply the LU direct solver on each of the two preconditioning blocks.

The number of iterations required to solve the system for the different meshes at hand is reported in Table 1. We can see that the block-diagonal preconditioner is optimal, namely the number of required iterations does not depend on the mesh.

- 4 To investigate the spectra of the original and preconditioned systems, we consider a smaller problem, e.g. with $n=5$. Using the forms defined above, we can extract a NumPy array with the entries of the matrices Σ and P , and since the problem is small, it is affordable to compute the inverse of P :

```
Sigma = assemble(a, bcs=bcs).M.values
P = assemble(myPCform, bcs=bcs).M.values
prec_Sigma = np.linalg.inv(P)*Sigma
```

By the function `np.linalg.eig`, we can compute the eigenvalues of Σ and $P^{-1}\Sigma$: plotting them on the real line (both the matrices are real and symmetric, thus their eigenvalues are all real numbers), we notice how the eigenvalues of the preconditioned matrix are much more clustered than those of Σ , thus confirming the suitability of the preconditioner. Further inspection on the number of eigenvalues contained in each cluster verifies the spectral properties of the system that were discussed in the lessons.

- 5 To use the suggested block triangular preconditioner, we modify the preconditioning form as follows, before passing it to the `LinearVariationalProblem`:

```
pc_form = inner(grad(u), grad(v)) * dx + p * q * dx + div(u) * q * dx
```

Changing the form is not all, however. Indeed, if we keep `"pc_fieldsplit_type": "additive"`, only the diagonal blocks of the preconditioner are considered and therefore we get back to the case of point 3. To consider also the new lower-triangular block, we modify this parameter as `"pc_fieldsplit_type": "multiplicative"`. The results are reported in Table 1: the preconditioner is optimal (i.e. the number of GMRES iterations is independent of the discretization parameter h) and the number of iterations is significantly lower than the block-diagonal case. However, applying a block-triangular preconditioner is more costly, so each of these GMRES iterations requires more computational time. At the end, for this particular differential problem, the total computational time of solving the problem is practically the same in the two cases of points 3 and 5.